Milestone 4: Relational Schema and Data Access Architecture

Representing Film Casting and Production Networks

Karan Brahmbhatt
University of Victoria
Canada
karanbrahmbhatt@uvic.ca

Moksh Patel
University of Victoria
Canada
mokshpatel@uvic.ca

Abstract

This report details the logical design, implementation, and architectural decoupling of a relational database system designed to represent film casting and production networks. To ensure system reproducibility, security, and maintainability, the application utilizes a strict three-tier architecture, completely isolating the Presentation Layer (Flask UI) from the Data Access Layer (DAL). This approach guarantees secure, dynamic data manipulation while strictly enforcing Entity-Relationship (ER) constraints via SQLite PRAGMA directives.

1. PROBLEM DEFINITION

Our goal is to allow producers and casting directors to retrieve historical collaborations, active production slates, and cast networks. Flat lists of actors or movies lose the critical context of performance roles. We therefore link each casting action to one movie, one actor, and specific role attributes (e.g., Lead, Cameo). The database must answer queries regarding talent rosters, project portfolios, and multi-relational casting ledgers, ensuring robust data integrity throughout all modifications.

2. PROPOSED RELATIONAL MODEL

The logical schema translates the Conceptual Entity-Relationship (ER) model into Third Normal Form (3NF) compliant structures.

2.1. Mapping the IS-A Hierarchy

The system models Talent using an IS-A hierarchy. The superclass `Person` holds shared attributes (e.g., `Name`, `Id`), while the `Actor` and `Director` subclasses maintain one-to-one identifying relationships with `Person`. This structural integrity is enforced using candidate keys (`UNIQUE`) on the subclass foreign keys.

2.2. Referential Integrity and Cascading Deletions

Multi-relational data requires robust cleanup logic to prevent orphaned records. We implemented strict `ON DELETE CASCADE` constraints across all foreign keys. For example, the bridge entity `PerformanceRole` maps the Many-to-Many relationship between `Movie` and `Actor`. If a `Movie` is deleted, the SQLite engine automatically purges all associated casting records in `PerformanceRole` and `MovieDirector`, ensuring the database never retains references to non-existent projects.

3. RELATIONAL OPERATIONS

DML functionality is verifiable via the UI or the terminal by executing `python main.py --queries`, which outputs the results of multi-join aggregations, subqueries, and cascading deletion tests defined in `03_queries.sql`. Multi-step user actions (e.g., adding an actor and casting them in a movie simultaneously) are translated into atomic DAL transactions. It raises explicit `BizError` exceptions if business rules or integrity constraints are violated.

4. ARCHITECTURE AND USER INTERFACE

A cornerstone of this architecture is the strict separation of concerns (Figure 1). Raw SQL execution is entirely abstracted and must never occur within the presentation or business logic layers.

4.1. The Data Access Layer (DAL)

The Data Access Layer is isolated within the `src/dal` directory. It contains domain-specific modules (`movie_dal.py` and `person_dal.py`) responsible for executing parameterized DML statements (INSERT, UPDATE, DELETE, SELECT). By centralizing database connections within `database.py`, we ensure that critical settings, such as `PRAGMA foreign_keys = ON`, are consistently applied to every transaction. The DAL prevents SQL injection vulnerabilities by utilizing parameterized query binding.

4.2. Presentation Layer

The Presentation Layer (`src/ui/app.py`) provides an interactive HTML/CSS dashboard tailored for directors and producers. It captures user input and routes it to the Business Layer (`src/biz/services.py`), acting as the sole interface for database manipulation.

5. REPRODUCIBILITY AND EVALUATION

Early iterations relied on working-directory-dependent pathing, which introduced instantiation challenges across different terminal environments (e.g., GitHub Codespaces versus local IDEs).

This limitation was resolved by implementing a dynamic relative pathing protocol in `src/cmn/config.py`. By utilizing Python's `__file__` variable, the application dynamically calculates paths relative to the script's physical location in the repository, ignoring the user's terminal execution state. Consequently, the application is fully reproducible: executing

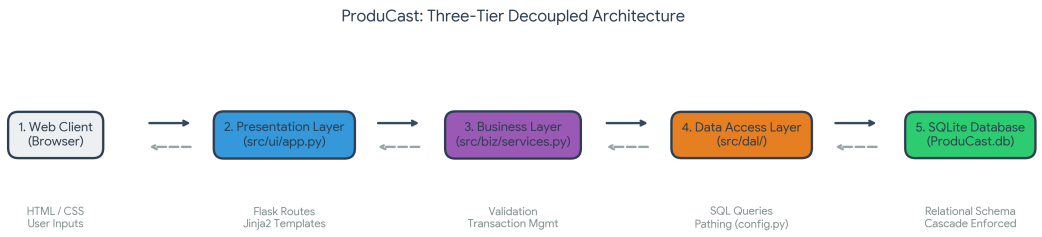


Figure 1: The Three-Tier Architecture highlighting the distinct separation between the Flask UI, Business Logic (biz), and the Data Access Layer (DAL).

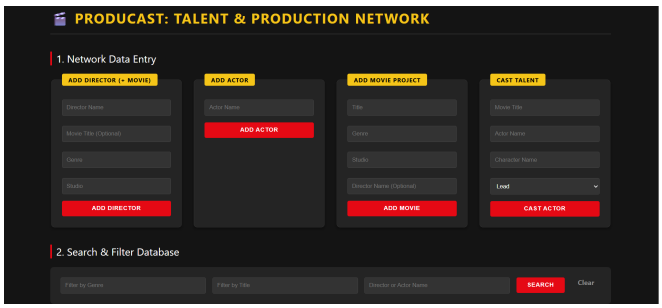


Figure 2: Network Data Entry: Abstracting multi-table inserts.

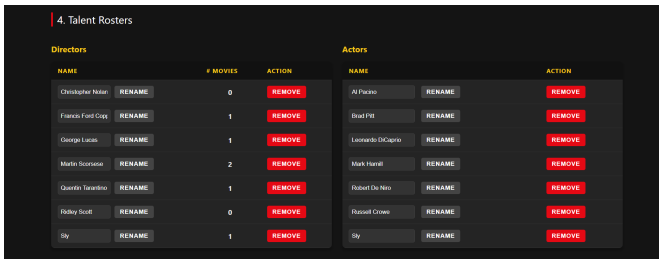


Figure 4: Talent Rosters: Managing the `Person` IS-A hierarchy.

`python main.py` from the repository root will reliably bootstrap the schema, seed the data, and launch the user interface seamlessly across any host environment. The complete source code, relational DDL, seed data, and detailed reproduction instructions have been packaged into the accompanying repository for evaluation.

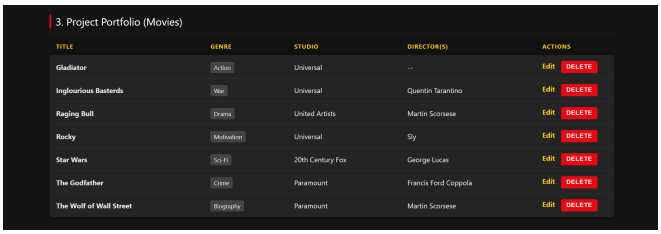


Figure 3: Project Portfolio: Displaying active `Movie` relations.

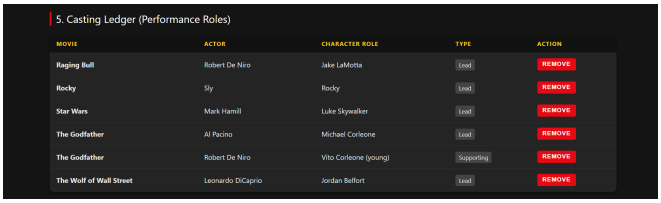


Figure 5: Casting Ledger: The `PerformanceRole` bridge table.